

Centro Federal de Educação Tecnológica de Minas Gerais

Engenharia de Computação

Computação Gráfica

Relatório de Desenvolvimento

Desafio do Engenheiro de Satélites Espaciais

João Paulo da Cunha Faria

Maria Eduarda Teixeira Souza

Divinópolis - MG

Setembro de 2026

1 Introdução

Neste desafio da Aula 12, a proposta foi montar um satélite formado por quatro peças: um corpo, dois painéis solares e uma antena. Todas as peças foram fornecidas modeladas na origem (0,0), no Sistema de Referência do Objeto (SRO). O corpo e os painéis usam o mesmo quadrado, de vértices $(-1,-1)$, $(1,-1)$, $(1,1)$ e $(-1,1)$, e a antena é um triângulo de vértices $(-1,0)$, $(1,0)$ e $(0,2)$.

O objetivo era aplicar transformações de Translação, Rotação e Escala (TRS) em cada peça para levá-las até a posição correta no Sistema de Referência do Universo (SRU), chegando à imagem do satélite apresentada em aula. Este relatório explica como o código em *Python* foi construído para resolver o problema.

2 Fundamentação

Para juntar translação, rotação e escala em uma única operação, foram usadas coordenadas homogêneas, como visto em aula: cada ponto (x, y) ganha uma terceira coordenada igual a 1 e passa a ser escrito como $[x,\ y,\ 1]^T$. Assim, as três transformações viram matrizes 3×3:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1}$$

Com isso, basta multiplicar as três matrizes para obter uma matriz única para cada peça:

$$M = T \cdot R \cdot S \qquad \Rightarrow \qquad P_{SRU} = M \cdot P_{SRO} \tag{2}$$

A ordem importa. A matriz é aplicada da direita para a esquerda, então a peça é primeiro escalada, depois rotacionada e só no final transladada. Como a escala e a rotação acontecem em relação à origem, elas precisam ser feitas enquanto a peça ainda está em (0,0). Se a translação viesse antes, a peça seria deslocada junto com a rotação e a escala e sairia do lugar.

3 Metodologia

O código completo está no arquivo `desafio_satelite.py`. Nesta seção ele é apresentado por inteiro, na ordem em que aparece no arquivo, e cada parte é explicada logo após a sua listagem. A numeração das linhas nas listagens é a mesma do arquivo original.

3.1 Materiais e Configuração

O código foi desenvolvido em *Python* com duas bibliotecas: `NumPy`, para criar as matrizes e fazer as multiplicações, e `Matplotlib`, para desenhar as peças no plano cartesiano. A instalação é feita com:

```
pip install numpy matplotlib
```

No início do arquivo, as duas bibliotecas são importadas com os apelidos usuais `np` e `plt` (Listagem 1).

Listagem 1: Importação das bibliotecas

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

3.2 Funções das Transformações

As três transformações da Equação 1 foram implementadas como funções (Listagem 2). Cada uma recebe os parâmetros da transformação e devolve a matriz 3×3 correspondente em coordenadas homogêneas:

- `matriz_translacao(tx, ty)`: coloca os deslocamentos t_x e t_y na terceira coluna da matriz identidade;
- `matriz_escala(sx, sy)`: coloca os fatores de escala na diagonal principal;
- `matriz_rotacao(angulo_graus)`: recebe o ângulo em graus e o converte para radianos com `np.radians`, que é a unidade esperada por `np.cos` e `np.sin`. O seno e o cosseno são calculados uma única vez e reaproveitados na montagem da matriz.

O tipo `np.float32` foi usado em todas as matrizes para que os valores decimais do seno e do cosseno não sejam arredondados para inteiros.

Listagem 2: Funções que geram as matrizes de translação, escala e rotação

```
def matriz_translacao(tx, ty):
    # Matriz de Translação em Coordenadas Homogêneas
    return np.array([
        [1, 0, tx],
        [0, 1, ty],
        [0, 0,  1]
    ], dtype=np.float32)


def matriz_escala(sx, sy):
    # Matriz de Escala em Coordenadas Homogêneas
    return np.array([
        [sx,  0, 0],
        [0,  sy, 0],
        [0,   0, 1]
    ], dtype=np.float32)


def matriz_rotacao(angulo_graus):
    # Matriz de Rotação em torno da origem (plano XY)
    theta = np.radians(angulo_graus)
    cos_t, sin_t = np.cos(theta), np.sin(theta)

    return np.array([
        [cos_t, -sin_t, 0],
        [sin_t,  cos_t, 0],
        [0,           0, 1]
    ], dtype=np.float32)
```

3.3 Funções de Desenho

Duas funções auxiliares cuidam da parte visual (Listagem 3).

A função `desenhar_peca` recebe uma matriz de vértices e separa a primeira linha (coordenadas X) e a segunda linha (coordenadas Y). Com elas, `ax.plot` desenha o contorno da peça e `ax.fill` pinta o seu interior com transparência (`alpha=0.25`), para que seja possível ver as peças mesmo quando estão sobrepostas. A terceira linha (W) não é usada no desenho, pois ela só serve para as contas com as matrizes.

A função `configurar_plano` prepara cada gráfico: define o título e os nomes dos eixos, limita a área visível a $x \in [-8, 8]$ e $y \in [-6, 6]$, desenha os eixos $x = 0$ e $y = 0$ em preto, adiciona a grade e a

legenda. O comando `ax.set_aspect('equal')` mantém a mesma escala nos dois eixos, sem o qual os quadrados poderiam aparecer como retângulos e os ângulos de rotação ficariam distorcidos na imagem.

Listagem 3: Funções auxiliares de desenho e configuração do plano cartesiano

```
35 def desenhar_peca(ax, matriz, cor, rotulo):
36     # Cada coluna da matriz é um vértice [X, Y, W]^T
37     x = matriz[0, :]
38     y = matriz[1, :]
39
40     ax.plot(x, y, color=cor, linestyle='-', linewidth=2, label=rotulo)
41     ax.fill(x, y, color=cor, alpha=0.25)
42
43
44 def configurar_plano(ax, titulo):
45     # Plano Cartesiano (Grade, Eixos, Limites)
46     ax.set_title(titulo, fontsize=13, fontweight='bold')
47     ax.set_xlabel("Eixo X", fontsize=11)
48     ax.set_ylabel("Eixo Y", fontsize=11)
49
50     # limites da janela de visualização (Viewport)
51     ax.set_xlim(-8, 8)
52     ax.set_ylim(-6, 6)
53     ax.set_aspect('equal')
54
55     # Desenhando os eixos X=0 e Y=0 em negrito
56     ax.axhline(0, color='black', linewidth=1.5)
57     ax.axvline(0, color='black', linewidth=1.5)
58
59     # Configurando a grade
60     ax.set_xticks(np.arange(-8, 9, 2))
61     ax.set_yticks(np.arange(-6, 7, 2))
62     ax.grid(True, linestyle=':', color='gray', alpha=0.7)
63     ax.legend(loc='upper right', fontsize=9)
```

3.4 Geometria das Peças

A função principal, `desafio_satelite`, começa definindo as peças no SRO (Listagem 4). Elas são guardadas como matrizes em que cada coluna é um vértice $[X, Y, W]^T$, com $W = 1$. O primeiro vértice é repetido no final para que o polígono seja desenhado fechado: por isso o quadrado tem 5 colunas e o triângulo tem 4.

Organizar os vértices em colunas permite que uma única multiplicação `M @ vertices` transforme a peça inteira de uma vez. O corpo e os dois painéis usam a mesma matriz `quadrado_unitario`, já que no SRO eles são o mesmo quadrado; o que muda entre eles são apenas as transformações. Em seguida, as duas matrizes são impressas no terminal para registro.

Listagem 4: Definição dos vértices das peças no SRO em coordenadas homogêneas

```
66 def desafio_satelite():
67     print("=== Desafio do Engenheiro de Satélites Espaciais ===\n")
68
69     # Definição da Geometria Original (SRO - Sistema de Referência do Objeto)
70     # Todas as peças foram modeladas na origem (0,0).
71     # Cada coluna é um vértice: [X, Y, W]^T e o último vértice fecha o polí
           gono.
72
73     # Corpo e Painéis: p1(-1,-1), p2(1,-1), p3(1,1), p4(-1,1)
74     quadrado_unitario = np.array([
75         [-1,  1, 1, -1, -1],  # Linha do Eixo X
```

```
        [-1, -1, 1,  1, -1],  # Linha do Eixo Y
        [1,   1, 1,  1,  1]   # Linha do Eixo W (Coordenada Homogênea)
    ], dtype=np.float32)

    # Antena: p1(-1,0), p2(1,0), p3(0,2)
    triangulo_antena = np.array([
        [-1, 1, 0, -1],  # Linha do Eixo X
        [0,  0, 2,  0],  # Linha do Eixo Y
        [1,  1, 1,  1]   # Linha do Eixo W (Coordenada Homogênea)
    ], dtype=np.float32)

    print("Quadrado Unitário no SRO (corpo e painéis):")
    print(quadrado_unitario, "\n")

    print("Triângulo da Antena no SRO:")
    print(triangulo_antena, "\n")
```

3.5 Composição das Matrizes TRS

Os valores de cada transformação foram escolhidos observando a imagem esperada do satélite (Tabela 1):

- **Corpo:** o quadrado é apenas dobrado de tamanho, passando de 2×2 para 4×4, e continua centrado na origem;
- **Painéis:** o quadrado é esticado 3 vezes no eixo x e reduzido à metade no eixo y, virando uma barra de 6×1. Depois, o painel esquerdo é girado $30°$ e levado para $(-4, 0)$, e o direito é girado $-30°$ e levado para $(4, 0)$;
- **Antena:** o triângulo é reduzido à metade e movido 2 unidades para cima. Como a base do triângulo já está em $y = 0$, ela fica exatamente em $y = 2$, apoiada no topo do corpo.

Tabela 1: Valores de escala, rotação e translação usados em cada peça.

Peça	**Escala** (s_x, s_y)	**Rotação**	**Translação** (t_x, t_y)
Corpo Principal	(2; 2)	0°	(0; 0)
Painel Esquerdo	(3; 0,5)	30°	(−4; 0)
Painel Direito	(3; 0,5)	−30°	(4; 0)
Antena	(0,5; 0,5)	0°	(0; 2)

Na Listagem 5, a matriz de cada peça é montada com o operador `@` (multiplicação de matrizes do NumPy), sempre na ordem $T \cdot R \cdot S$ da Equação 2. Mesmo quando uma transformação não é necessária, como a rotação do corpo e da antena ou a translação do corpo, ela foi mantida com valor neutro (ângulo 0 ou deslocamento $(0, 0)$). Assim todas as peças seguem o mesmo padrão, e alterar uma peça depois exige apenas trocar um valor.

Por fim, as peças são reunidas na lista `matrizes`. Cada item guarda o nome da peça (usado na legenda), a sua matriz composta, a geometria original a ser transformada e a cor usada no desenho.

Listagem 5: Composição das matrizes TRS de cada peça e lista de peças

```
    # Composição das Matrizes TRS (Translação * Rotação * Escala)
    # Dica: Ordem da Direita para a Esquerda! (M_T * M_R * M_S)
    # Primeiro a peça é dimensionada, depois girada e só então posicionada.

    # 1) Corpo: quadrado 2x2 -> 4x4, centrado na origem
    M_corpo = (matriz_translacao(0, 0)
```

```
                @ matriz_rotacao(0)
                @ matriz_escala(2, 2))

    # 2) Painel Esquerdo: vira uma barra 6x1, inclinada +30° e levada para (-4, 0)
    M_painel_esq = (matriz_translacao(-4, 0)
                    @ matriz_rotacao(30)
                    @ matriz_escala(3, 0.5))

    # 3) Painel Direito: mesma barra, inclinada -30° e levada para (4, 0)
    M_painel_dir = (matriz_translacao(4, 0)
                    @ matriz_rotacao(-30)
                    @ matriz_escala(3, 0.5))

    # 4) Antena: reduzida à metade e encaixada no topo do corpo (0, 2)
    M_antena = (matriz_translacao(0, 2)
                @ matriz_rotacao(0)
                @ matriz_escala(0.5, 0.5))

    matrizes = [
        ("Corpo Principal", M_corpo, quadrado_unitario, 'gray'),
        ("Painel Esquerdo", M_painel_esq, quadrado_unitario, 'blue'),
        ("Painel Direito", M_painel_dir, quadrado_unitario, 'blue'),
        ("Antena", M_antena, triangulo_antena, 'red')
    ]
```

3.6 Aplicação das Transformações e Renderização

A última parte da função (Listagem 6) cria uma figura com dois gráficos lado a lado usando o comando `plt.subplots(1, 2)`. O gráfico `ax_sro`, à esquerda, é usado para as peças originais, e o gráfico `ax_sru`, à direita, para o satélite montado.

O laço `for` percorre a lista `matrizes` e, para cada peça:

1. imprime a matriz composta no terminal, arredondada para 4 casas decimais;
2. calcula os vértices da peça no SRU com a linha `vertices_sru = matriz_composta @ vertices_sro`. Essa linha é exatamente a operação $P_{SRU} = M \cdot P_{SRO}$ da Equação 2, aplicada a todos os vértices de uma vez;
3. imprime os vértices resultantes no terminal;
4. desenha a peça original no gráfico da esquerda e a peça transformada no gráfico da direita.

Depois do laço, os dois gráficos são configurados com `configurar_plano`. O `plt.tight_layout()` ajusta os espaçamentos da figura e o `plt.show()` abre a janela com o resultado. A legenda é criada dentro de `configurar_plano`, e por isso essa função só é chamada depois que todas as peças já foram desenhadas.

Listagem 6: Aplicação das matrizes e desenho das peças no SRO e no SRU

```
    # Renderização 2D com o Matplotlib
    fig, (ax_sro, ax_sru) = plt.subplots(1, 2, figsize=(16, 7))

    for rotulo, matriz_composta, vertices_sro, cor in matrizes:
        print(f"Matriz Composta (T * R * S) - {rotulo}:")
        print(np.round(matriz_composta, 4), "\n")

        # Multiplicação Matricial: leva a peça do SRO para o SRU
        vertices_sru = matriz_composta @ vertices_sro
```

```
133
134         print(f"Vértices de {rotulo} no SRU:")
135         print(np.round(vertices_sru, 4), "\n")
136
137         desenhar_peca(ax_sro, vertices_sro, cor, rotulo)
138         desenhar_peca(ax_sru, vertices_sru, cor, rotulo)
139
140     configurar_plano(ax_sro, "1. Peças Modeladas na Origem (SRO)")
141     configurar_plano(ax_sru, "2. Cena Montada: Satélite no SRU")
142
143     print("[*] Renderizando o plano cartesiano...")
144     plt.tight_layout()
145     plt.show()
```

3.7 Execução do Programa

As duas últimas linhas do arquivo (Listagem 7) fazem com que a função `desafio_satelite` seja executada apenas quando o arquivo é rodado diretamente (`python desafio_satelite.py`), e não quando ele é importado por outro script.

Listagem 7: Ponto de entrada do programa

```
148 if __name__ == "__main__":
149     desafio_satelite()
```

4 Resultados

A Figura 1 mostra o resultado da execução. À esquerda, as peças estão todas sobrepostas na origem, como foram fornecidas. À direita, o satélite aparece montado e igual à imagem apresentada em aula.

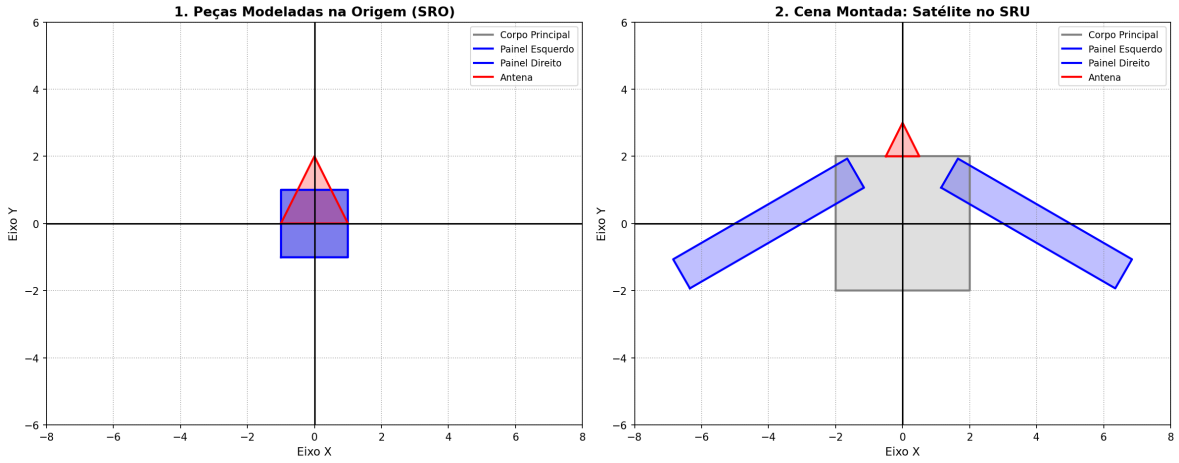


Figura 1: Peças na origem (SRO) e satélite montado após as transformações (SRU).

Os valores impressos no terminal confirmam as posições esperadas: o corpo vai de -2 a 2 nos dois eixos, a antena tem base em $y = 2$ e ponta em $(0, 3)$, e os painéis ficam simétricos em relação ao eixo y. Por exemplo, as matrizes compostas dos dois painéis foram:

$$M_{painel_esq} = \begin{bmatrix} 2{,}598 & -0{,}25 & -4 \\ 1{,}5 & 0{,}433 & 0 \\ 0 & 0 & 1 \end{bmatrix} \qquad M_{painel_dir} = \begin{bmatrix} 2{,}598 & 0{,}25 & 4 \\ -1{,}5 & 0{,}433 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

A única diferença entre elas é o sinal dos termos ligados ao seno e à translação em x, o que faz sentido, já que um painel é o espelho do outro.

Para visualizar melhor a ordem das transformações, a Figura 2 mostra o painel esquerdo depois de cada etapa. Primeiro a escala transforma o quadrado em uma barra; depois a rotação inclina a barra, ainda na origem; e só no final a translação leva o painel para o lugar certo.

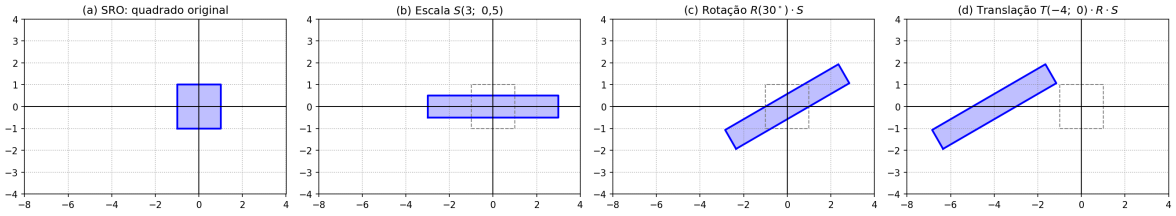


Figura 2: Painel esquerdo após cada etapa: (a) original, (b) escala, (c) rotação e (d) translação. O tracejado mostra o quadrado original.

5 Conclusão

O desafio mostrou na prática como montar uma cena a partir de peças simples modeladas na origem. Com apenas um quadrado e um triângulo, e mudando só os valores de escala, rotação e translação, foi possível montar o satélite completo.

O uso de coordenadas homogêneas permitiu juntar as três transformações em uma única matriz por peça, o que deixou o código simples: cada peça precisa de apenas uma multiplicação para ir do SRO para o SRU. Também ficou claro que a ordem $T \cdot R \cdot S$ é importante, porque a escala e a rotação precisam acontecer enquanto a peça ainda está na origem.

Referências

[1] RODRIGO, Raulivan. **Transformações Gráficas: Transformações Geométricas**. Notas de Aula (08 a 12 - Transformações Gráficas.pdf). Departamento de Computação, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), 2026.

[2] NUMPY DEVELOPERS. **NumPy Documentation**. Disponível em: https://numpy.org/doc/stable/.

[3] MATPLOTLIB DEVELOPMENT TEAM. **Matplotlib: Visualization with Python**. Disponível em: https://matplotlib.org/stable/.